

Practice Contents

- Step 1: Download the Files
 - a) Open a terminal window and log in to your Linux VM.
 - b) Use the following commands to download the files:

```
wget www.nku.edu/~foxr/CIT371/employees.txt
wget www.nku.edu/~foxr/CIT371/students.txt
```
- Step 2: Computing Payroll Information
 - a) Write a script to compute payroll information for the people in the *employees.txt* file. The script will use a while read statement. Each row of this file consists of a person's last name, hours, and wages. To compute pay, use an if-then-else statement using this formula:
 - Hours ≤ 40 --> pay = hours * wages
 - Hours > 40 --> pay = 40 * wages + 2 * (hours - 40) * wages
 - b) We can't use time and a half (1.5) for overtime, so use 2. Output for each employee their pay and also output the total number of employees processed and the average pay computed.

```
#!/bin/bash
```

```
TOTAL_EMPLOYEES=0
```

```
TOTAL_PAY=0
```

```
while read -r LAST_NAME HOURS WAGES; do
```

```
  if [ -z "$LAST_NAME" ]; then
```

```
    continue
```

```
  fi
```

```
  if [ $HOURS -le 40 ]; then
```

```
    PAY=$((HOURS * WAGES))
```

```
  else
```

```
    OVERTIME=$((HOURS - 40))
```

```
    PAY=$((40 * WAGES + 2 * OVERTIME * WAGES))
```

```
  fi
```

```
  echo "Employee: $LAST_NAME | Pay: \$$PAY"
```

```
  TOTAL_PAY=$((TOTAL_PAY + PAY))
```

```
  TOTAL_EMPLOYEES=$((TOTAL_EMPLOYEES + 1))
```

```
done
```

```
if [ $TOTAL_EMPLOYEES -gt 0 ]; then
```

```
  AVERAGE_PAY=$((TOTAL_PAY / TOTAL_EMPLOYEES))
```

```
  echo "-----"
```

```
  echo "Total employees processed: $TOTAL_EMPLOYEES"
```

```
  echo "Average pay: \$$AVERAGE_PAY"
```

```
fi
```

Cách chạy: ./payroll.sh < employees.txt

- Step 3: Finding Students by Major

- a) Write a script that receives a parameter of a major and uses a while read statement to input the data from the *student.txt* file. Output a list of all students who are of that major and output the number of students found. For instance, you might call this with `./script16 CSC < student.txt` to find all CSC students.

```
#!/bin/bash
```

```
TARGET_MAJOR=$1
COUNT=0
```

```
if [ -z "$TARGET_MAJOR" ]; then
    echo "Error: Please provide a major as a parameter."
    exit 1
fi
```

```
while read -r LAST_NAME FIRST_NAME MAJOR REST; do
    if [ "$MAJOR" == "$TARGET_MAJOR" ]; then
        echo "$FIRST_NAME $LAST_NAME"
        COUNT=$((COUNT + 1))
    fi
done
```

```
echo "-----"
echo "Total students found in $TARGET_MAJOR: $COUNT"
```

Cách chạy: `./find_major.sh CSC < students.txt`

- Step 4: Assigning Letter Grades

- a) Write a script that uses a while read statement to input all of the data, line by line, computing the average test score for the student and then assigning a letter grade. See figure 6.16 in the textbook for an example of what the nested if-else statement should look like. The 5th test is worth double (this will require adding it twice and then dividing the total by 6 instead of 5). Output for each student the student's last name, major, and letter grade. Additionally, sum up the number of students who passed (got at least a D or higher) and the number of students who failed. When you run your script, remember to redirect input from *students.txt*.

```
#!/bin/bash
```

```
PASSED=0
FAILED=0
```

```
while read -r LAST_NAME MAJOR T1 T2 T3 T4 T5; do
    if [ -z "$LAST_NAME" ]; then
        continue
    fi
```

```
    TOTAL_SCORE=$((T1 + T2 + T3 + T4 + (T5 * 2)))
    AVG=$((TOTAL_SCORE / 6))
```

```
    if [ $AVG -ge 90 ]; then
        GRADE="A"
```

```

elif [ $AVG -ge 80 ]; then
    GRADE="B"
elif [ $AVG -ge 70 ]; then
    GRADE="C"
elif [ $AVG -ge 60 ]; then
    GRADE="D"
else
    GRADE="E/F"
fi

echo "Student: $LAST_NAME | Major: $MAJOR | Grade: $GRADE"

if [ "$GRADE" == "E/F" ]; then
    FAILED=$((FAILED + 1))
else
    PASSED=$((PASSED + 1))
fi
done

echo "-----"
echo "Students Passed: $PASSED"
echo "Students Failed: $FAILED"

```

Cách chạy: `./grades.sh < students.txt`

- Step 5: Converting Permissions to Numeric Equivalents

- a) Write a script that is passed a list of items (files, directories, links, etc) in the current directory as parameters. The script will iterate through this list, obtaining the long listing for each item and convert the permissions of the item into the 3-digit numeric equivalent. To accomplish this, you will have to store the permission portion in a variable. Assume the current filename is stored in the variable FILE. You can obtain the permissions using the following instruction:

```
permissions=`ls -l $FILE | awk '{print $1}'`
```

- b) The variable permissions will store the 11 characters in the permissions (the type of object, - for file, the 9 characters that make up the permissions, and a period that ends this sequence). You want to access characters 2-10. To access a single character, you can use notation like this `${permissions:INDEX:1}` where INDEX is a number between 1 and 9 (recall that for this notation, the first character is at index 0). Now you want to look at the 3 sets of characters (1-3, 4-6, 7-9) and convert the r, w, and x into a value. You will need three separate sums, we can for instance call them first, second and third. If the character at index 1 is 'r', then add 4 to first, otherwise do nothing. If the character at index 2 is 'w', then add 2 to first, otherwise do nothing. And so forth. You might have if-then instructions that look like this:

```

if [ ${permissions:1:1} == r ]
then first=$((first+4))
fi

```

- c) Once done, output the name of the file and the values of first, second, and third. For instance:
`addresses.txt 744.`

```
#!/bin/bash
```

```
PASSED=0
```

```
FAILED=0
```

```
while read -r LAST_NAME MAJOR T1 T2 T3 T4 T5; do
    if [ -z "$LAST_NAME" ]; then
        continue
    fi

    TOTAL_SCORE=$((T1 + T2 + T3 + T4 + (T5 * 2)))
    AVG=$((TOTAL_SCORE / 6))

    if [ $AVG -ge 90 ]; then
        GRADE="A"
    elif [ $AVG -ge 80 ]; then
        GRADE="B"
    elif [ $AVG -ge 70 ]; then
        GRADE="C"
    elif [ $AVG -ge 60 ]; then
        GRADE="D"
    else
        GRADE="E/F"
    fi

    echo "Student: $LAST_NAME | Major: $MAJOR | Grade: $GRADE"

    if [ "$GRADE" == "E/F" ]; then
        FAILED=$((FAILED + 1))
    else
        PASSED=$((PASSED + 1))
    fi
done

echo "-----"
echo "Students Passed: $PASSED"
echo "Students Failed: $FAILED"
```

- Step 6: Determining Palindromes

- a) Write a script to determine if a parameter supplied to the script is a palindrome or not. A palindrome is a string that reads the same forward and backward. To accomplish this, you will need to loop from 0 to the halfway point in the string and compare the character at location i to the character at location $\text{length} - i - 1$ where length is the length of the string. If, after passing halfway through, the two compared characters match in each iteration, then output the parameter and that it is a palindrome, else output the parameter and that it is not a palindrome.

```
#!/bin/bash
```

```
STR=$1
```

```
if [ -z "$STR" ]; then
```

```

    echo "Error: Please provide a string parameter."
    exit 1
fi

LENGTH=${#STR}
HALF=$((LENGTH / 2))
IS_PALINDROME=1

for ((i=0; i<HALF; i++)); do
    CHAR_FRONT=${STR:$i:1}
    CHAR_BACK=${STR:$((LENGTH - i - 1)):1}

    if [ "$CHAR_FRONT" != "$CHAR_BACK" ]; then
        IS_PALINDROME=0
        break
    fi
done

if [ $IS_PALINDROME -eq 1 ]; then
    echo "$STR is a palindrome"
else
    echo "$STR is not a palindrome"
fi

```

Cách chạy: ./palindrome.sh racecar hoặc ./palindrome.sh hello

- Step 7: Creating a Palindrome Function
 - a) Create a script containing a function of the code defined in #5 and code after the function that defines an array of several strings, some of which are palindromes and some of which are not and then code to iterate through the array, calling the palindrome function passing each array element as a parameter.

```
#!/bin/bash
```

```

check_palindrome() {
    local STR=$1
    local LENGTH=${#STR}
    local HALF=$((LENGTH / 2))
    local IS_PALINDROME=1

    for ((i=0; i<HALF; i++)); do
        local CHAR_FRONT=${STR:$i:1}

```

```

    local CHAR_BACK=${STR:${(LENGTH - i - 1):1}}

    if [ "$CHAR_FRONT" != "$CHAR_BACK" ]; then
        IS_PALINDROME=0
        break
    fi
done

if [ $IS_PALINDROME -eq 1 ]; then
    echo "$STR is a palindrome"
else
    echo "$STR is not a palindrome"
fi
}

STRING_ARRAY=("racecar" "hello" "madam" "linux" "noon" "radar" "bash")

for ELEMENT in "${STRING_ARRAY[@]}"; do
    check_palindrome "$ELEMENT"
done
Cách chạy: ./palindrome_func.sh

```